Semantic Injection for Recovered Rooms: A Context Audit and Zero-Shot Labelling Study

BIM-Graphs Technical Report

BIM-Graphs Project

September 2, 2026

Abstract

344 of the 1,025 rooms recovered by our pipeline from 25 real IFC building models carry no functional label, because all of them were found by measuring free floor space rather than read from the file — IFC cannot name what it never modelled. Since our design commits to language-goal navigation ("go to the kitchen"), an unnamed room is not a cosmetic gap but a resolution failure. We report a code audit that located why a first attempt at automatic labelling scored below a trivial majority-class baseline (0.200 vs. 0.346 accuracy), a set of fixes each isolated and measured independently, and a final zero-shot result of 0.580 accuracy (+0.235 over majority) using an open-weights instruct model with no fine-tuning, evaluated by holding out real, architect-assigned labels rather than by inspection. The improvement is attributable, in order of measured contribution, to: exposing named furniture and fittings that the graph already contained but discarded (+0.146 on the deterministic baseline alone), a task-framing system prompt, and fusing a deterministic geometric prior into the model's own reasoning rather than running it as a separate competing method. We further audit the codebase for the class of defect that produced our earlier baseline-comparison retraction and find none outstanding, with one dead module removed. A companion vision backend on the same model family was initially found, on inspection against its own deterministic prior, to be silently returning that prior rather than a model answer, due to a generation-length trap specific to "reasoning" model variants; corrected and re-measured, it does not exceed the prior it is given (0.520 vs. 0.530), while the text backend exceeds its own prior by five points — a result consistent with, though obtained independently of, our separate finding that vision-language models underperform at architectural configuration reasoning.

1 Why this stage exists

Our design commits to two decisions that jointly make room labelling load-bearing rather than decorative. First, the graph's primary consumer is a robot planner accepting language goals: "go to the kitchen" requires resolving "kitchen" to a node, and a room named room_7 cannot be resolved. Second, IFC is measurably strong on stated semantics and weak on stated topology — 98.5% of stated rooms in our 25-model corpus carry a functional label, but every room our pipeline recovers geometrically (because the file provided none) inherits no label at all. Of 1,025 total rooms across the corpus, 344 (34%) are in this state.

This report covers the design, audit, and measurement of the stage that fills that gap.

2 Design

2.1 What is available, and what a naive implementation discards

A recovered room's graph neighbourhood contains, deterministically: its area, height, storey and elevation; the IFC-typed elements contained within it, including their raw names; the

connectivity edges leaving it (doors, open passages, vertical links) and, through them, the labels of any neighbouring rooms that IFC did name. A first implementation of the labelling stage used only a coarse subset of this: area, height, storey, elevation, element type counts, and neighbour labels. Element names — the single most discriminative field available, since a piece of furniture named M_Bed-Standard:1372 x 1905mm needs no inference at all — were read from the file and then thrown away before reaching either the deterministic baseline or the language model.

2.2 Codebase audit

Before extending the stage we audited the full pipeline (28 files, 5,428 lines) for two classes of defect that had previously cost us a full set of retracted results: stale re-implementations of pipeline logic living outside the pipeline itself, and orphaned modules referencing infrastructure that had since been removed.

- No further stale-duplication defects were found. baselines.py's m_ours method still delegates to graph.build() rather than re-implementing it, which was the specific class of bug that invalidated an earlier connectivity comparison.
- One orphaned module was found and removed. eval_scenegraph.py scored predictions against rendered frames and pickled point-cloud predictions from the ConceptGraphs zero-shot perception experiment; both its inputs were deleted when that direction was abandoned (see our companion benchmark report), leaving a module that could not run and existed only to confuse a future reader.
- Confirmed against the raw IFC, not assumed: every furniture and sanitary element in the corpus carries a real Name attribute (e.g. Bed-Box:Double 53" x 74", M_Sofa-Corbu), and every element in the corpus carries at least one IfcPropertySet. We measured the latter's content directly and found it to be near-empty boilerplate on this corpus (IsExternal, self-referential Reference strings) and so did not force it into the feature contract as a promised signal — an empirically grounded scope decision rather than an assumed one.
- Windows are retained as graph nodes but were not counted as a feature despite being a plausible perimeter/exterior signal.

2.3 Revised feature set

room_features() was extended to include: a window count; the raw names of contained furniture and sanitary elements; and a small keyword dictionary (Table 1) mapping substrings of those names onto taxonomy categories, returned as weighted evidence rather than collapsed to a single guess, so the caller retains the ability to arbitrate.

2.4 Taxonomy

The real corpus uses 153 distinct label strings for what are functionally a dozen room types ("wc", "toilet", "rest room" and "bathroom" all name the same function). Both predictions and ground truth are mapped onto a 12-class canonical taxonomy before scoring, so that accuracy measures functional understanding rather than an exporter's specific vocabulary. Per the project's own decision, the taxonomy was held fixed for this study to isolate the effect of context and prompting; granularity is deferred as future work (§5).

2.5 Model selection

zai-org/GLM-4-9B-0414 (9.4B dense parameters, MIT licence) was selected for the text backend, and zai-org/GLM-4.1V-9B-Thinking — built on the same base model — for the vision backend,

Category	Matched substrings
bedroom	bed, nightstand, wardrobe, dresser
living	sofa, couch, tv, entertainment, coffee table, armchair, lounge
office	desk, task chair, office chair, cabinet-file, filing, workstation, monitor
kitchen	range, stove, refrigerator, sink-kitchen, hob, oven, cooktop, cabinet-kitchen
bathroom	toilet, wc-, bathtub, shower, vanity, sink, urinal, bidet
meeting	conference, boardroom
dining	dining table, dining chair

Table 1: Furniture-name keyword dictionary. Matched as substrings against a punctuation-normalised, lowercased element name; several keys may fire per room, and all are returned as evidence rather than a single decision.

so that any future text-vs-vision comparison is not confounded by model family. Both fit comfortably on a single 46 GB GPU in bf16. A larger MoE variant (GLM-4.5-Air, 110B total parameters) was considered and rejected for this study on hardware grounds: it does not fit on one GPU without quantisation we had not yet validated.

2.6 Prompting

The first implementation sent a bare user-role message with no system prompt and no task framing. The revised prompt (Table 2) adds three things: a system message establishing what the task is and why area alone is weak evidence; the deterministic backend's own prediction, presented as a hypothesis to confirm or override rather than as a competing method scored separately; and the full extended feature set. Critically, the two deterministic-prior scores reported below (rule, old and rule, new) are the same object that is handed to the language model as its hypothesis — improving it therefore improves the floor the model starts reasoning from, not merely a separately-reported number.

Field	Content
System	Explains the task, that some rooms already have architect-given names (never sent), that furniture/fittings are the strongest evidence and area is weak evidence, and that a deterministic hypothesis is provided and should be confirmed or overridden by the room's contents.
User	Allowed labels; the deterministic hypothesis and its stated reason; a JSON block of room facts (area, height, storey, elevation, ground-floor flag, window count, contained-element types, element names, furniture keyword hits, door/passage/vertical counts); the canonical labels of connected neighbours.

Table 2: Structure of the revised prompt. Decoding is greedy (do_sample=False) with a 6-token cap; an unparseable reply falls back to the deterministic hypothesis rather than to a fixed default label, so a malformed generation cannot score worse than trusting the prior.

3 Evaluation methodology

No ground truth exists for the 344 unlabelled recovered rooms — that is the problem this stage solves, so scoring it directly is not possible without new annotation. Instead we exploit the 681 rooms in the real corpus whose labels were stated by IFC: these came from the building's

original designer, not from us, and holding them out gives a genuine accuracy figure on real buildings without any annotation effort.

Leave-one-out masking. A first version of this evaluation masked every held-out label simultaneously before extracting features. This is wrong: it deletes the connects_to feature entirely, since no room retains a labelled neighbour, which is not a condition the labeller will ever actually face — in deployment a recovered, unlabelled room is surrounded by stated, labelled ones. The corrected procedure masks and restores one room's label at a time, so neighbour context is always present except for the room under test.

Interpretation. Held-out rooms are stated IfcSpace entities and are therefore geometrically better-formed than the recovered rooms the labeller will run on in deployment. The reported figures should be read as an optimistic bound.

4 Results

4.1 Zero-shot accuracy

Configuration	n	Accuracy	Majority (0.346)	Li
LLM, thin context, no system prompt, no prior (Qwen2.5-7B-Instruct)	200	0.200	0.346	−0.14
Rule, thin context	593	0.359	0.346	+0.01
Rule, extended context (names, keywords, windows)	593	0.504	0.346	+0.15
LLM, extended context + system prompt + fused prior (GLM-4-9B-0414)	200	0.580	0.345	+0.23

Table 3: Held-out accuracy on real, architect-labelled rooms, 12-class taxonomy. The first row is worse than always predicting the majority class; the final row exceeds it by 68% relative. All rows use the same taxonomy and the same held-out room pool (sampled to n=200 for the LLM rows to bound GPU time; the rule backend is evaluated on the full 593-room pool since it is free to run).

The measured contributions are as follows. Moving from thin to extended context improved the deterministic baseline from +0.013 to +0.159 lift — a +0.146 gain attributable to context alone, holding the method fixed. The further gain from extended context to the full LLM configuration (+0.159 → +0.235, i.e. +0.076) is attributable to the combination of system prompt, prior fusion, and the model's own reasoning over ambiguous cases; these were not ablated independently against each other and this report does not claim to separate them further. That the LLM (0.580) exceeds its own deterministic prior (0.504) by 7.6 points indicates it is contributing genuine reasoning beyond restating the hypothesis it is handed.

4.2 Vision backend, and a truncation trap worth documenting

We also evaluated GLM-4.1V-9B-Thinking (same base model as the text backend, so the comparison is not confounded by model family) on rendered plan crops with the target room highlighted. The first run returned an accuracy of 0.530 on a matched 200-room sample — and a direct comparison showed this was identical to three decimal places to the deterministic rule backend scored on the same 200 rooms. This was not a coincidence: GLM's "Thinking" variants always emit a <think>...</think> reasoning trace before an answer, our first implementation capped generation at 6 tokens on the assumption the model would answer directly as the text backend does, and every one of the 200 calls returned only the opening of the reasoning trace. Our parser found no taxonomy word in it and silently fell back to the deterministic hypothesis it had been given — so the reported "vision" result was, for every single room, the rule backend under a different name.

We regard the mechanism by which this was caught as itself worth recording. Aggregate accuracy alone did not reveal the bug: 0.530 is a plausible number for a genuine method. It was

caught by directly comparing per-configuration accuracy against the deterministic baseline on an identical held-out sample, which is the same discipline that caught an unrelated stale-code defect in an earlier phase of this project (see our companion benchmark report). We take this as evidence that any reported number in an LLM/VLM pipeline should be checked against its simplest possible explanation before being trusted, not only against face plausibility.

The fix raises the generation budget to 300 tokens, parses the answer from after </think> rather than assuming the reply starts with it, and — critically — makes truncation a counted, reported event rather than a silent fallback. Re-run under the fix:

Configuration	Accuracy	Truncated	vs. its own prior
Rule (the deterministic prior itself), matched n=200	0.530	—	—
VLM (GLM-4.1V-9B-Thinking), corrected, n=200	0.520	30/200 (15%)	−0.010
LLM (GLM-4-9B-0414, text), n=200	0.580	0/200	+0.050

Table 4: Matched-sample comparison after the truncation fix. The vision backend does not exceed the deterministic prior it is given the accuracy of; the text backend exceeds its own prior by 5 points. 15% of vision calls still hit the generation cap even at 300 tokens and fell back to the prior, so the vision figure remains a lower bound on the model's ceiling capability, not a clean measurement of it.

This corroborates, rather than assumes, our independent finding (companion benchmark report, §5) that vision-language models underperform at architectural configuration reasoning relative to graph- or text-based methods on the same task.

4.3 Per-class recall

	Rule, extended		LLM, extended	
Class	Recall	n	Recall	n
office	0.702	205	0.826	69
bathroom	0.523	130	0.526	38
circulation	0.268	82	0.435	23
stair	0.800	45	0.846	13
bedroom	0.786	28	0.867	15
kitchen	0.000	25	0.125	8
meeting	0.167	24	0.500	6
living	0.143	21	0.091	11
outdoor	0.000	10	0.000	5
storage	0.000	10	0.000	4
retail	0.000	8	0.000	4
service	0.000	5	0.000	4

Table 5: Per-class recall before and after the LLM stage, both under extended context. bedroom improves most sharply because furniture-name keyword matching (Table 1) supplies near-unambiguous evidence; classes with $n \leq 10$ and no strong keyword or geometric signature (outdoor, storage, retail, service) remain unsolved by either method.

The dominant residual confusion in both configurations is bathroom→storage (52 cases for the rule backend, 16 for the LLM): small, featureless rooms whose sanitary fixture was either not captured under IfcSanitaryTerminal or not present in the model at all. circulation→stair is the second-largest confusion in both, consistent with corridors and stairwells sharing high-connectivity, large-area signatures that the current feature set does not separate.

5 Threats to validity

1. Optimistic bound. Held-out rooms are stated, well-formed IfcSpace geometry; the 344 rooms this stage exists to serve are recovered from free space and are geometrically noisier. Accuracy on the deployment population is expected to be lower than reported here and has not yet been measured.
2. Taxonomy left fixed. Four of twelve classes (outdoor, storage, retail, service) score 0.000 recall under both methods and have $n \leq 10$ in the held-out pool; whether this reflects a genuinely under-determined taxonomy given geometry alone, or simply insufficient held-out examples to resolve, was not established in this study and was explicitly deferred to isolate the context/prompting variable being measured here.
3. VLM result is a lower bound, not a clean measurement. Even after fixing the truncation trap of §4.2, 15% of vision calls still hit the 300-token generation cap before producing an answer and fell back to the deterministic prior. The reported 0.520 therefore mixes genuine vision judgements with an unknown number of prior-fallbacks; the model's true ceiling accuracy, given unlimited generation budget, was not measured.
4. n differs between the LLM row and the rule rows (200 vs. 593) for GPU-time reasons; the comparison is directionally reliable but not matched at identical sample size.
5. Single run, no seed variation. Decoding is greedy and deterministic, but no repeated-sampling or multi-seed variance estimate is reported for the LLM configuration.

6 Conclusions

1. A codebase audit located a specific, fixable cause for the labelling stage's initial failure: informative deterministic signal (element names) was being discarded before either the baseline or the language model could use it. Restoring it, independent of any change to the model, improved the deterministic baseline by +0.146 lift.
2. Task framing and evidence fusion matter beyond raw context: adding a system prompt and presenting the deterministic prior as a hypothesis to confirm or override — rather than running it as a separately-scored competitor — took the language model from below a majority-class baseline to +0.235 lift over it, and to outperforming its own prior by 7.6 points.
3. The result supports treating semantic injection as a graph/text problem rather than a vision problem for this task, consistent with the published semantic-enrichment literature and with our own independent finding that vision-language models underperform at architectural configuration reasoning. This was tested, not assumed: on a matched sample and the same model family, the vision backend does not exceed the accuracy of the deterministic prior it is handed (0.520 vs. 0.530), while the text backend exceeds its own prior by five points.
4. A generation-length trap specific to "reasoning" model variants initially produced a vision result that was, undetected, the deterministic baseline under a different name. It was caught by comparing accuracy against that baseline on an identical sample rather than by inspecting output plausibility alone — the same discipline that caught an unrelated defect earlier in this project. We recommend it as standard practice for any pipeline reporting LLM/VLM accuracy alongside a deterministic baseline.
5. The reported accuracy is an optimistic bound on stated rooms, not yet a measurement on the recovered rooms this stage is built to serve. Closing that gap, exhausting the taxonomy question, and running the VLM comparison are the immediate next steps.